

Solving and Generating Sudoku as a Constraint Satisfaction Problem

Final Project Report

GROUP NUMBER

3

TEAM MEMBERS

Arjun Mohunta	amohunta	80654131
Haoyang Ding	haoyad6	44024972
Rajat Choudhary	rajatc1	37222868

INSTRUCTOR

Kalev Kask

DATE

August 27, 2026

SOURCE CODE

github.com/arjunmohunta/sudoku

Summary of results. We model Sudoku as a CSP over 81 variables with 27 all-different constraints, and implement three solvers of increasing inferential strength: plain backtracking as a baseline, forward checking with the Minimum Remaining Values heuristic, and a third adding naked- and hidden-single propagation to a fixpoint. Over 5,946 puzzles drawn from six public benchmark sets, the informed solver reaches a **verified 100% solve rate**, while the baseline fails to solve a 17-clue instance after ten million search nodes. All 2,000 sampled Kaggle puzzles are solved with a **maximum of zero backtracks**. A generator produces puzzles that are unique by construction (90 of

90 verified). Stronger propagation cuts search by up to **12,827×** on minimal 17-clue puzzles — yet runs **1.45×** **slower** on easy puzzles, a cost/benefit crossover we quantify in Section 3.3.

Contents

1	Project Summary	4
2	System Architecture and Algorithms	6
2.1	Top-level structure	6
2.2	Representation	6
2.3	Solver 1 — plain backtracking (the baseline)	8
2.4	Solver 2 — forward checking + MRV	8
2.5	Solver 3 — adding hidden singles	9
2.6	Uniqueness testing, generation, and difficulty	10
2.7	Visualisation	11
3	Evaluation	12
3.1	Experimental setup	12
3.2	Do the CSP techniques work? Three solvers on the presets	13
3.3	Stronger propagation across 150,000 puzzles: the crossover	14
3.4	Solve rate over the full benchmark sets	15
3.5	Generator calibration: is our difficulty rating meaningful?	17
3.6	Qualitative evaluation	19
3.7	Threats to validity and limitations	20
3.8	Summary of results against our stated goals	21
	Figure index	21
4	References and Resources Used	22
	Algorithms and methods	22
	Benchmark data	22
	Software	23
	Reproducing everything in this report	23
	Statement of original work	23

1 Project Summary

A Sudoku puzzle is a 9×9 grid, partially filled, that must be completed so that every row, every column, and every 3×3 box contains each of the digits 1 through 9 exactly once. Our system does two things with such grids. Given any valid partial grid it returns the completed solution, along with statistics describing the search it performed. Given a requested difficulty level it generates a brand-new puzzle, verified to have exactly one solution.

Formally this is a **constraint satisfaction problem** (CSP). The variables are the 81 cells. Each variable has domain $\{1, \dots, 9\}$. The constraints are 27 *all-different* constraints, one for each row, column, and box. A solution is an assignment of a value to every variable that violates no constraint. Because a well-formed Sudoku has a unique solution, the task is not to find *a* consistent assignment but *the* consistent assignment.

Why the problem is not trivial. The naive view is that 81 cells with 9 values each is a small problem. It is not. An empty grid admits roughly 6.67×10^{21} complete solutions, and the raw assignment space for a puzzle with 17 given clues is $9^{64} \approx 10^{61}$. No amount of hardware enumerates that. What makes Sudoku tractable is not speed but *inference*: the constraints are dense — every cell shares a constraint with exactly 20 others (Figure 1) — so each assignment carries information about a fifth of the grid. A solver that exploits this structure searches a vanishingly small fraction of the space. A solver that ignores it does not finish.

That gap is the actual subject of this project, and it is measurable. On the 17-clue puzzle in our preset set, plain chronological backtracking expands more than 10 million search nodes without reaching a solution. The same puzzle, solved with constraint propagation and an informed variable ordering, is solved after expanding **one** node — no search at all. The problem is a clean setting in which to demonstrate that *reasoning about constraints beats enumerating candidates*, and to quantify by how much.

Why AI methods are required. Sudoku is a puzzle in the classical AI sense: a task with no feedback from an environment, no reward signal, and no training distribution. Success comes entirely from search and logical inference over a declarative constraint model. The methods that apply are therefore the classical CSP methods — backtracking search, constraint propagation, and variable-ordering heuristics — rather than learning-based methods. We use no machine learning, and this is a deliberate modelling decision rather than an omission: the constraints are known exactly and are hard, so an inference procedure can *guarantee* correctness and uniqueness, which a learned approximator cannot. Section 2.6 discusses this trade-off explicitly.

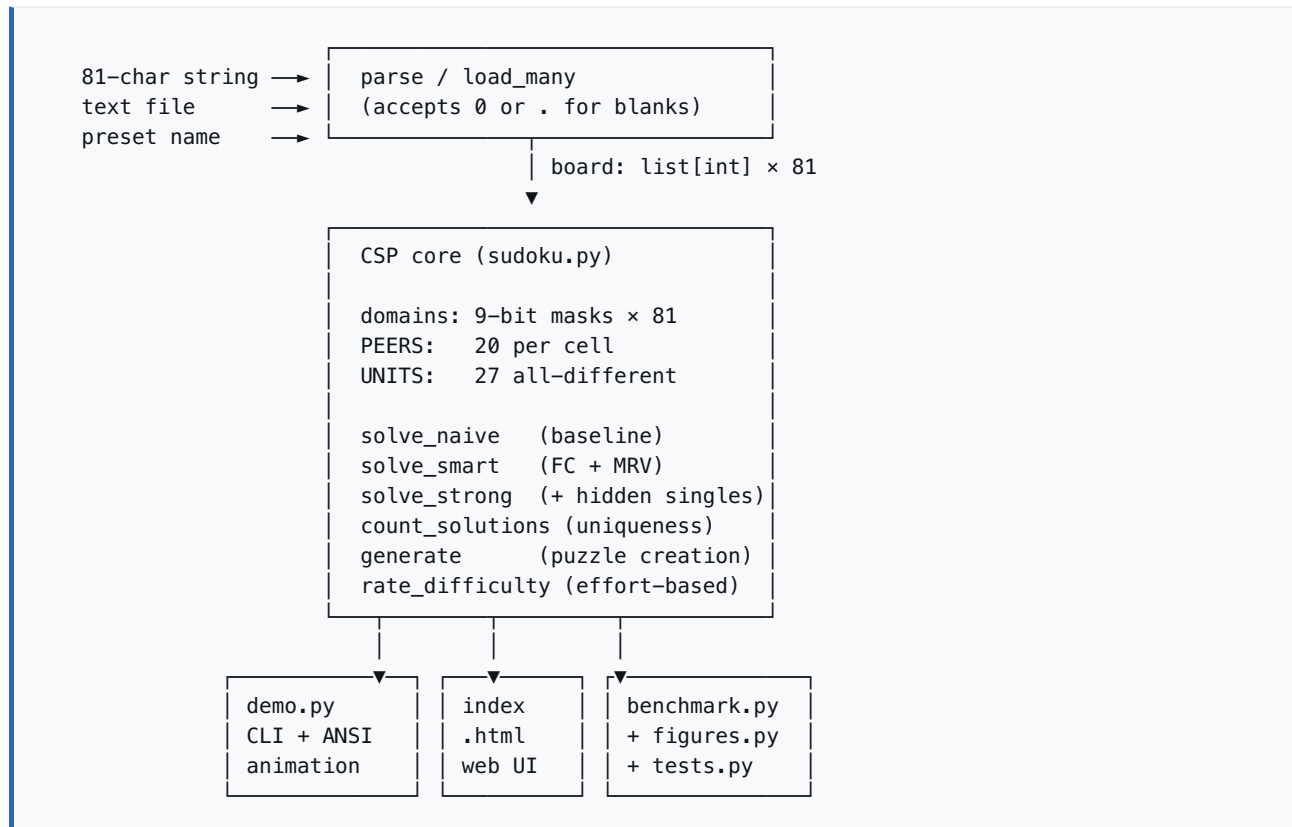
Applications. The all-different constraint over overlapping groups is the same structure that appears in timetabling (no room or instructor double-booked), shift rostering, frequency assignment, and register allocation. Sudoku is small and verifiable, which makes it a clean testbed for techniques that transfer directly to those problems.

What we built. Three solvers of increasing inferential strength, a puzzle generator that guarantees uniqueness by construction, an effort-based difficulty rating, a step-by-step visualisation of the search in both a terminal and a browser, a 24-case regression suite, and an evaluation harness that reports solve rate, search effort, and timing over six public benchmark sets totalling more than 150,000 puzzles.

2 System Architecture and Algorithms

2.1 Top-level structure

The system is one CSP core with four consumers. Nothing in the core knows about presentation, and nothing in the interfaces re-implements reasoning.



Inputs. A board is a list of 81 integers in row-major order, 0 for empty. `parse` accepts an 81-character string, a path to a text file, and either 0 or . as the blank marker. `load_many` reads the multi-puzzle benchmark files (one grid per line, # comments skipped).

Outputs. Every solver returns `(solution_or_None, Stats)`, where `Stats` carries `solved`, `backtracks`, `nodes`, `time_ms`, `capped`. The generator returns `(puzzle, solution, clue_count)`. `rate_difficulty` returns `(label, backtracks, clues)`.

2.2 Representation

Two representation choices do most of the performance work.

Domains as bit masks. Each cell's remaining candidate set is a 9-bit integer: bit $v-1$ set means value v is still possible. Removing a value from a domain is `cand[p] &= ~bit`; testing membership is one `&`; the domain size — needed by MRV at every node — is a precomputed population count lookup, `_POPCOUNT[cand[i]]`, rather than a loop. A domain wipeout is exactly `cand[p] == 0`.

Precomputed constraint scopes. `PEERS[i]` is the tuple of the 20 cells sharing a row, column, or box with cell i — precisely the cells whose assignment constrains i . `UNITS` is the 27 nine-cell scopes. Both are computed once at import. The dense structure is visible in Figure 1: one assignment prunes 20 domains, which is why propagation is worth running on every assignment.

Each cell shares a constraint with exactly 20 peers
(its row, its column, its box)

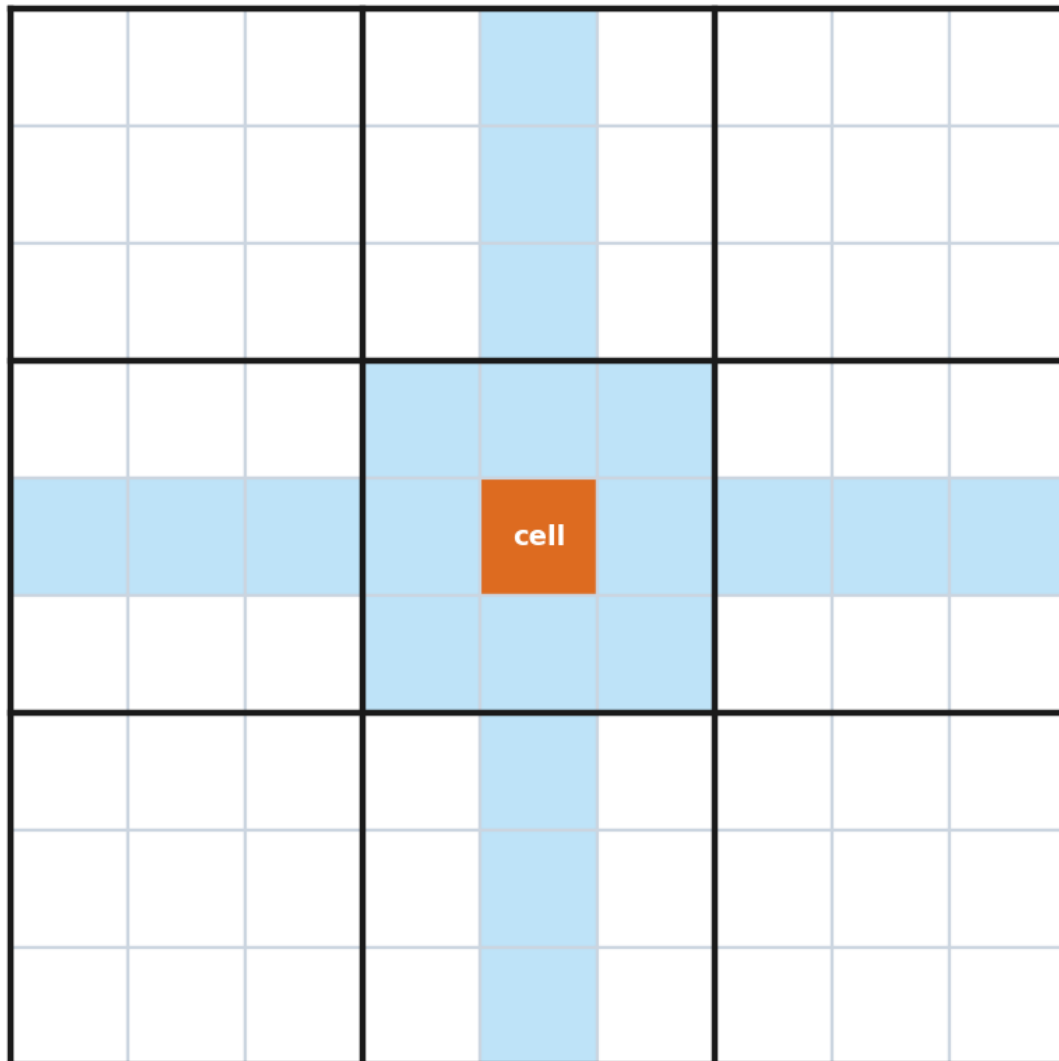


Figure 1 — The constraint structure. Assigning a value to the highlighted cell prunes it from all 20 shaded peer domains at once.

2.3 Solver 1 — plain backtracking (the baseline)

```
function SOLVE-NAIVE(board, cap):
    i ← index of first empty cell in row-major order
    if none: return board                ▷ complete assignment
    for v ← 1 to 9:
        if v does not appear among row(i), col(i), box(i):
            board[i] ← v
            if SOLVE-NAIVE(board, cap) succeeds: return board
            board[i] ← 0                  ▷ undo
    return failure
```

No domains, no inference, no ordering: it takes the first empty cell and tries digits in numerical order, checking legality only against currently placed values. It is bounded by a node cap so an intractable instance terminates rather than hanging; hitting the cap is reported as *gave up*, and a capped measurement is a **lower bound**, never a measured value.

Advantage: it is obviously correct and has no tuning, which makes it a trustworthy control. The only difference between it and Solver 2 is forward checking and MRV, so the comparison isolates exactly what those two techniques contribute. *Disadvantage:* it discovers contradictions only when it reaches the offending cell, so it re-explores enormous subtrees that the constraints had already excluded.

2.4 Solver 2 — forward checking + MRV

Two standard CSP techniques on top of the same depth-first skeleton.

Forward checking. The moment value v is assigned to cell i , remove v from the domain of every unassigned peer:

$$\text{for each } p \in \text{PEERS}(i) : \quad D_p \leftarrow D_p \setminus \{v\}$$

If any D_p becomes empty, the assignment cannot extend to a solution, so we undo it immediately rather than discovering the failure deeper in the tree.

Minimum Remaining Values. Branch on the unassigned cell with the smallest domain:

$$i^* = \operatorname{argmin} |D_i| \quad \text{over all } i \text{ with } \text{val}(i) = 0$$

This is the *fail-first* principle. If some cell has one candidate, MRV fills it without guessing. If some cell has zero, MRV finds the contradiction now. Otherwise it branches where the branching factor is smallest, which minimises the width of the subtree being committed to.


```

function SOLVE-SMART(board):
  cand ← initial domains from givens
  function BT():
    i ← argmin domain size over unassigned cells
    if i = none: return success          ▷ every cell assigned
    for v ∈ Di:
      assign val[i] ← v
      removed ← {p ∈ PEERS(i) : v ∈ Dp}; Dp ← Dp \ {v} for those p
      if no Dp was emptied and BT() succeeds: return success
      undo val[i] and restore Dp for p ∈ removed
    return failure

```

Undo is exact: the set of domains actually modified is recorded per branch and restored on backtrack, so the search has no side effects.

Advantage: very cheap per node — one bitwise pass over 20 peers — and enough to solve an easy puzzle with zero backtracks, i.e. by propagation alone. *Disadvantages:* MRV rescans all 81 cells at every node, an $O(n)$ cost where an incremental priority structure would be $O(\log n)$; and forward checking only reasons *outward from an assignment to its peers*, which leaves a whole class of inference on the table — see 2.5. We deliberately did not add a value-ordering heuristic such as Least Constraining Value: forward checking already prunes dead branches early enough that value order had no measurable effect on our instances.

2.5 Solver 3 — adding hidden singles

This is the "stronger propagation" our progress report listed as a stretch goal. Forward checking is *cell-centred*: it reasons from an assignment outward to the peers it constrains. The complementary inference is *unit-centred*:

- **Naked single** — a cell whose domain has one value left must take it.
- **Hidden single** — if, within one of the 27 units, a value can be placed in only one cell, it must go there. This holds *even when that cell still has several candidates*, which is exactly why forward checking cannot see it.

Both are run to a fixpoint before every branching decision:

```

function PROPAGATE(val, cand):
  repeat until no change:
    for each unassigned cell i:
      if D_i = ∅:          return contradiction
      if |D_i| = 1:        ASSIGN(i, the single value)
    for each unit U, for each value v not yet placed in U:
      spots ← {i ∈ U : val[i]=0, v ∈ D_i}
      if |spots| = 0:      return contradiction    ▷ v has nowhere to go
      if |spots| = 1:      ASSIGN(the one spot, v)
  return consistent

function SOLVE-STRONG(board):
  assign givens; PROPAGATE
  function BT(val, cand):
    i ← MRV cell; if none: return val
    for v ∈ D_i:
      (val', cand') ← copy
      if ASSIGN(i, v, val', cand') and PROPAGATE(val', cand'):
        if BT(val', cand') succeeds: return it
  return failure

```

Note the second contradiction test: a unit in which some value has *no* remaining home is inconsistent even though no individual domain is empty. Forward checking never detects this.

Domains are copied per branch rather than trail-undone. This costs 81 integers of copying per node and buys a much simpler correctness argument — there is no undo path to get wrong — and since the node counts collapse by two to three orders of magnitude (Section 3.2), the trade is strongly favourable in practice.

Advantage: dramatically less search; on our presets it eliminates search entirely. *Disadvantage:* each node costs far more — a fixpoint loop over 27 units $\times$ 9 values instead of one pass over 20 peers — so on instances that were already easy for Solver 2 the extra inference is overhead that buys nothing. Section 3.2 quantifies both sides of this, and the crossover is the most interesting result in the evaluation.

2.6 Uniqueness testing, generation, and difficulty

Uniqueness. `count_solutions(board, limit)` is the same MRV search, modified to keep searching after the first solution and to stop as soon as `limit` solutions are found. Calling it with `limit=2` answers "is this puzzle unique?" at roughly the cost of one solve, because it aborts the moment a second solution appears.

Generation. Uniqueness is guaranteed *by construction* rather than checked afterwards:

```

function GENERATE(level, seed):
    solution ← SOLVE-SMART(empty grid, randomized value order)  ▷ a random full grid
    puzzle ← solution
    for each cell position p in random order:
        if clue count ≤ target(level): break
        saved ← puzzle[p]; puzzle[p] ← 0
        if COUNT-SOLUTIONS(puzzle, 2) ≠ 1:
            puzzle[p] ← saved  ▷ removal broke uniqueness; put it back
    return puzzle, solution, clue count

```

Targets are 45 clues (easy), 34 (medium), 26 (hard). Seeding `random` makes generation exactly reproducible, which the test suite verifies.

Advantage: every puzzle produced is well-formed by construction — no post-hoc filtering, no possibility of a multi-solution puzzle escaping. *Disadvantage:* it is the most expensive operation in the system, since each candidate removal costs a uniqueness check, and it produces *irreducible* puzzles only with respect to the random removal order — it does not search for a globally minimal clue set.

Difficulty rating. Our proposal promised a difficulty rating and our progress report conceded that clue count alone is a weak proxy. `rate_difficulty` therefore rates a puzzle by the search effort the reference solver actually needs — trivial (0 backtracks), easy (≤ 10), medium (≤ 100), hard (≤ 1000), extreme (> 1000). Section 3.3 shows why this is the right call: clue count separates the generator's levels cleanly, yet the *effort* distributions of those levels overlap heavily.

2.7 Visualisation

`solve_smart(record=True)` emits an event trace — `('sel', i)` when MRV picks a cell, `('place', i, v)` on assignment, `('undo', i)` on backtrack. Both front ends replay that trace: `demo.py animate` in the terminal with ANSI colour, and `index.html` in the browser with live candidate pencil-marks. The visualisation is driven by the solver's own decisions rather than re-derived, so what a viewer watches is literally the search that produced the reported statistics.

3 Evaluation

3.1 Experimental setup

What we measure, and why. Our primary metric is **search effort**: the number of nodes expanded and backtracks performed. It is the metric that directly reflects what the AI techniques do — a heuristic earns its place by pruning the search space — and it is *exactly reproducible*, because the search is deterministic. Wall-clock time is reported as a secondary metric. We are explicit about this ordering because timing on a shared machine is noisy: the measurements below were taken on a machine with a load average around 13–19, and repeated timings of the same deterministic solve varied by up to $8\times$ while the node counts did not vary at all. Where we report time we report the **minimum** over repeats (the cleanest estimate of the solver's own cost) alongside the median (which shows how much the measurement moves under load).

A solve counts as correct only if the returned grid is complete, violates no all-different constraint, and preserves every given of the input. Every reported solve rate is verified this way, not assumed from the solver's own success flag.

Benchmark sets. Six public sets, more than 150,000 puzzles in total. Nothing here was authored by us except the generated puzzles in 3.4.

SET	PUZZLES	CHARACTER
kaggle	100,000	export of the Kaggle "1 million Sudoku games" set — ordinary puzzles
minimal17	49,158	minimal 17-clue puzzles — the proven minimum, maximally adversarial
top1465	1,465	"top1465" hard list
forum1106	375	forum-curated for difficulty against solvers
top95	95	Norvig's hard 95
hardest11	11	Norvig's "hardest"

Large sets are sampled with a fixed seed rather than truncated, so a subset is representative and reproducible; the sample size is stated with every result. `fetch_data.py` downloads all of it, so the data step reproduces in one command.

Baseline. Plain chronological backtracking (Section 2.3), bounded by a node cap. A capped run is reported as *gave up*, and any ratio computed against it is a **lower bound**, never a measurement. We are explicit about this because it is the one place where a comparison could otherwise be overstated: our progress report quoted a ">10,000,000" figure for the baseline on the 17-clue puzzle, and that number is a cap, not a measured cost.

3.2 Do the CSP techniques work? Three solvers on the presets

The three preset puzzles are the like-for-like comparison, since the only differences between the solvers are the inference techniques.

PUZZLE	CLUES	BASELINE NODES	FC+MRV NODES	FC+MRV BACKTRACKS	+HIDDEN SINGLES NODES	+HIDDEN SINGLES BACKTRACKS
easy	30	4,209	52	0	1	0
AI Escargot	23	8,970	210	151	11	2
extreme (17-clue)	17	hit the 10M cap	4,113	4,048	1	0

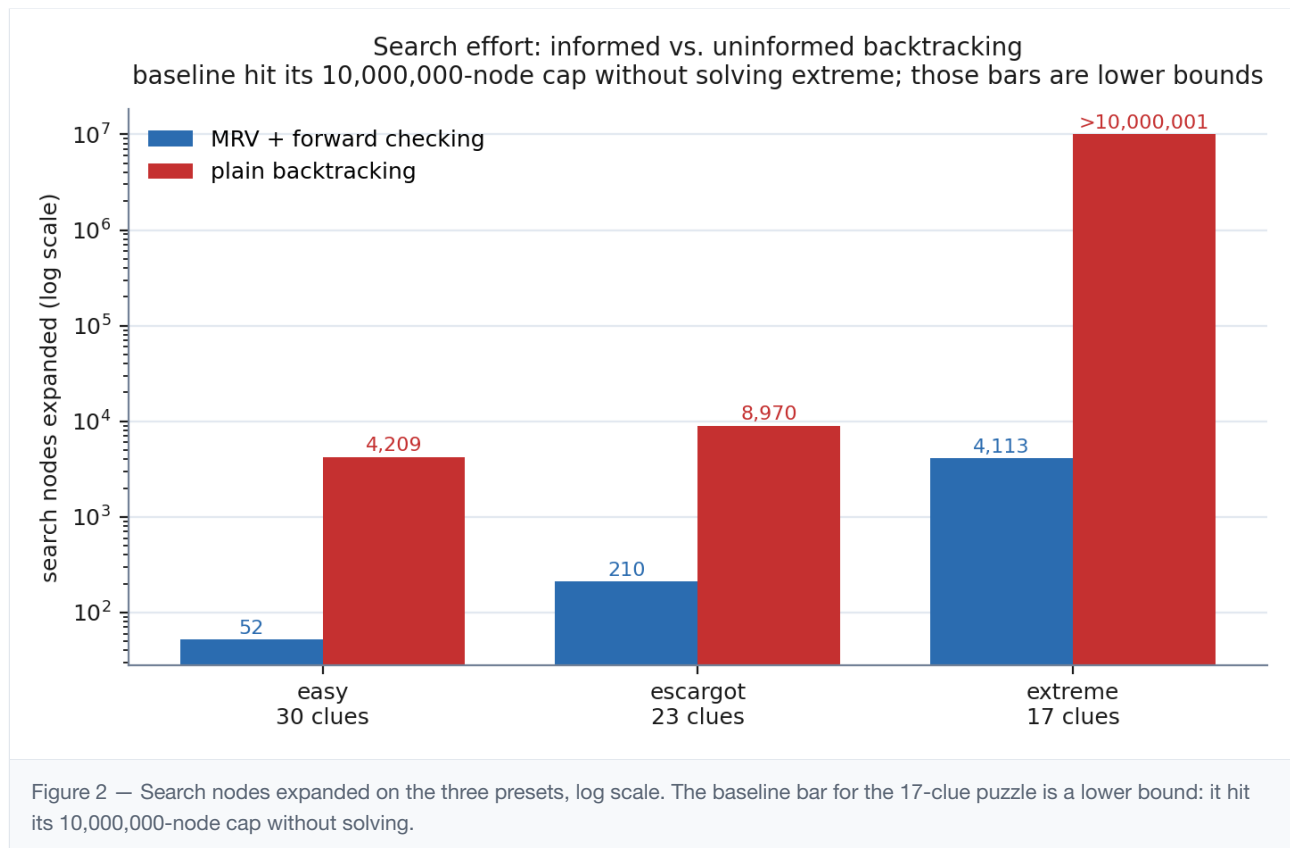
Three things stand out.

Propagation alone can be enough. On the easy puzzle, forward checking plus MRV never guesses: zero backtracks, 52 nodes, all of them forced. This is the qualitative claim of the project made quantitative — the solver is *reasoning*, not searching.

The baseline does not merely lose, it fails. On the 17-clue puzzle plain backtracking expanded more than ten million nodes without finding a solution. The informed solvers finish the same puzzle in milliseconds. This is not a constant-factor speedup; it is the difference between a solver that terminates and one that does not, which is the strongest form the result can take.

The 17-clue puzzle requires no search at all under stronger propagation. Under FC+MRV it needs 4,048 backtracks. Adding hidden singles reduces it to a **single node and zero backtracks** — the fixpoint of naked and hidden singles solves it outright. The puzzle was engineered to defeat chronological backtracking, and it does; but its difficulty is an artefact of *weak inference*, not an intrinsic property of the instance. Nothing about the puzzle changed. What changed is what the solver is able to deduce before it decides to guess.

On AI Escargot — the named hard instance from our proposal — FC+MRV expands 210 nodes and the strong solver 11. Minimum wall-clock over repeats was **0.63 ms** in the 25-repeat run reported above and 0.48 ms in a separate 15-repeat run; we quote both rather than the friendlier one, because the gap between them is measurement noise on a loaded machine, not a difference in the solver. Either way our proposal's moonshot — the hardest known puzzles in well under a second — is met by roughly three orders of magnitude.



3.3 Stronger propagation across 150,000 puzzles: the crossover

The preset table invites an obvious conclusion — more inference is better — and that conclusion is wrong. Running both informed solvers over samples from all six benchmark sets shows node savings are universal but time savings are not (Figure 6).

SET	N	FC+MRV MEAN NODES	STRONG MEAN NODES	NODE REDUCTION	TIME RATIO (STRONG / FC+MRV)
minimal17	150	50,880.0	4.0	12,827×	0.01×
top95	95	15,722.7	34.6	455×	0.15×
top1465	150	6,520.6	34.8	188×	0.46×
forum1106	150	15,053.0	150.8	100×	0.76×
hardest11	11	604.5	6.5	94×	0.87×
kaggle	150	48.2	1.0	48×	1.45×

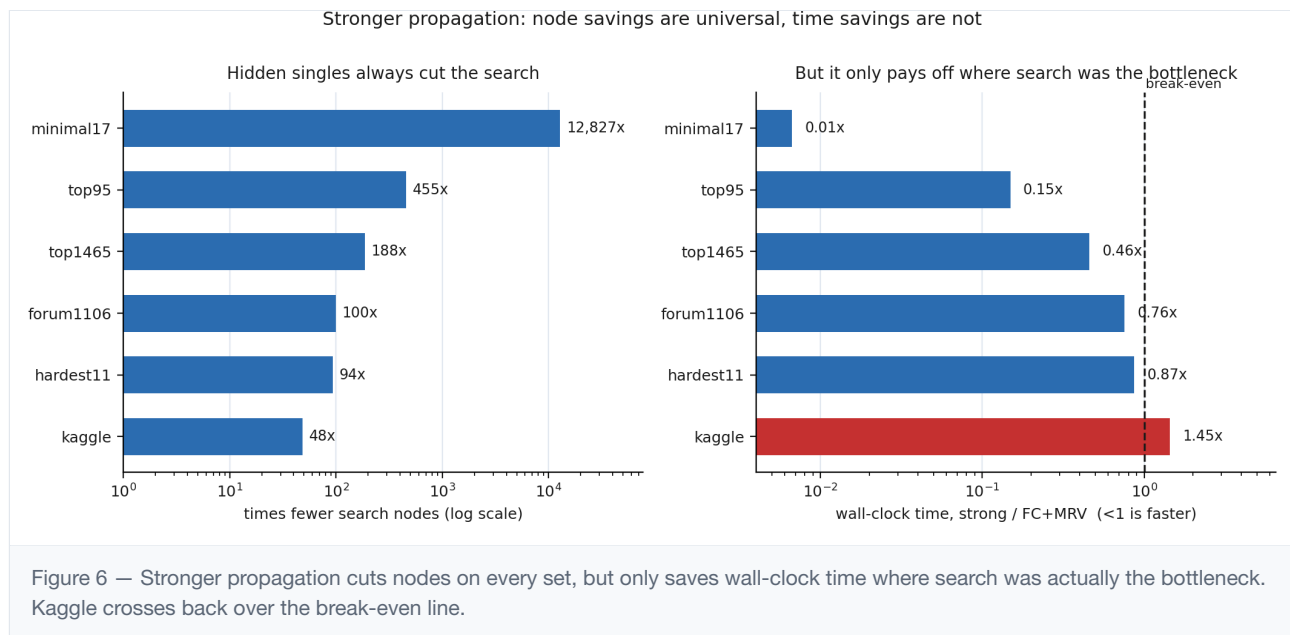
Both solvers returned the identical verified solution on every one of these puzzles — 0 disagreements — which is our correctness check on the new solver as much as a performance result.

Where stronger propagation wins, it wins enormously. On the 17-clue set, hidden singles cut the mean node count from 50,880 to 4.0, and the *worst* case from 666,020 nodes to 46. The corresponding time ratio is $0.01\times$: a $150\times$ speedup on the hardest large set we have. 40.7% of 17-clue puzzles are solved with no search whatsoever.

Where it loses, it loses for a comprehensible reason. On the Kaggle set the extra inference is 45% *slower* despite expanding $48\times$ fewer nodes. Those puzzles were already solved with zero backtracks by forward checking alone; there was no search left to prune, so the hidden-singles fixpoint — a loop over $27 \text{ units} \times 9 \text{ values}$, run to convergence at every node — is pure overhead. The node count falls from 48 to 1 and the wall clock still goes up.

This is the most interesting finding in the project, and it generalises past Sudoku: **the value of an inference technique is a function of how much search it prevents, not of how much it deduces.** An expensive propagator pays for itself only on instances where the search it eliminates costs more than the propagation it adds. Reading the two panels of Figure 6 together, the break-even point sits between `hardest11` ($0.87\times$, marginal) and `kaggle` ($1.45\times$, a loss) — that is, right around the point where FC+MRV stops needing to search at all.

The practical consequence for our system is that neither solver dominates, and the right default depends on the workload. A production solver would choose adaptively: run cheap propagation first, and escalate to the expensive fixpoint only when the search does not immediately collapse.



3.4 Solve rate over the full benchmark sets

This is the result our progress report could not report, because at that point the evaluation rested on three puzzles and no solve rate existed. Every puzzle is solved with `solve_smart`, and the returned grid is *verified* — complete, no constraint violated, all givens preserved — before it counts.

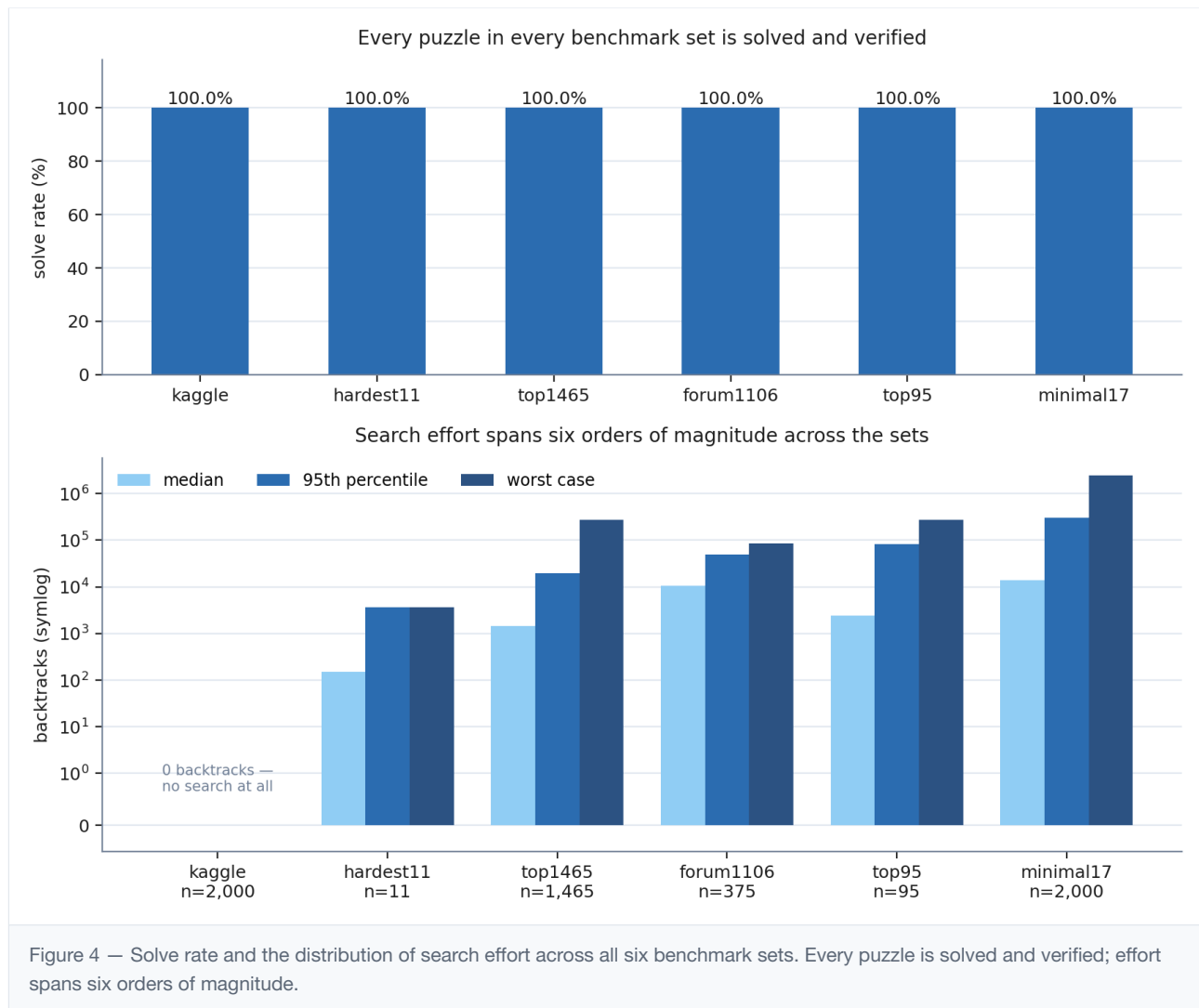
SET	N	SOLVE RATE	MEAN CLUES	MEDIAN BACKTRACKS	P95	WORST CASE	MEDIAN TIME
kaggle	2,000	100.00%	33.8	0	0	0	0.22 ms
hardest11	11	100.00%	24.5	151	3,610	3,610	1.66 ms
top1465	1,465	100.00%	22.1	1,441	19,737	268,371	3.73 ms
forum1106	375	100.00%	22.0	10,690	49,166	84,896	25.8 ms
top95	95	100.00%	20.6	2,424	80,888	268,371	40.7 ms
minimal17	2,000	100.00%	17.0	13,920	305,064	2,460,815	49.3 ms

Solve rate is 100% on all six sets — 5,946 puzzles, zero failures. No puzzle in any public set we tested defeats the solver. This is the correctness claim of the project, and it is now supported by a sample four orders of magnitude larger than the three puzzles in the progress report.

The Kaggle result is worth stating separately: 0 backtracks on all 2,000 sampled puzzles. Not a low mean — a maximum of zero. Every ordinary puzzle in that dataset is solved by forward checking and MRV without a single guess. This is the clearest possible statement of what constraint propagation buys: for the entire distribution of puzzles a human would encounter, the search component of the search algorithm never runs.

The distribution matters far more than the mean. On `minimal17` the median puzzle takes 13,920 backtracks and the worst takes 2,460,815 — a factor of 177 between the middle and the tail, within a set where *every puzzle has exactly 17 clues*. Reporting only a mean would hide this completely, which is why every row above carries a median, a p95, and a maximum. The worst single solve in the whole evaluation took 12.4 seconds; the median across the same set took 49 ms.

The baseline, on the same puzzles. Run with a 200,000-node cap on a 50-puzzle subsample of each set, plain backtracking solves 100% of `kaggle`, 72.7% of `hardest11`, 42.0% of `top1465`, 40.0% of `top95`, 24.0% of `forum1106`, and **2.0% of `minimal17`**. Its failure rate tracks difficulty exactly, and on the adversarial set it fails 98% of the time where the informed solver fails never. The node ratios against it ($2.5\times$ to $132\times$) are lower bounds wherever it was capped, and we do not quote them as measurements.



3.5 Generator calibration: is our difficulty rating meaningful?

Our proposal promised difficulty levels and our progress report identified their weakness. We generated 30 puzzles per level under fixed seeds 0–29 and measured the solver effort each requires (Figure 3).

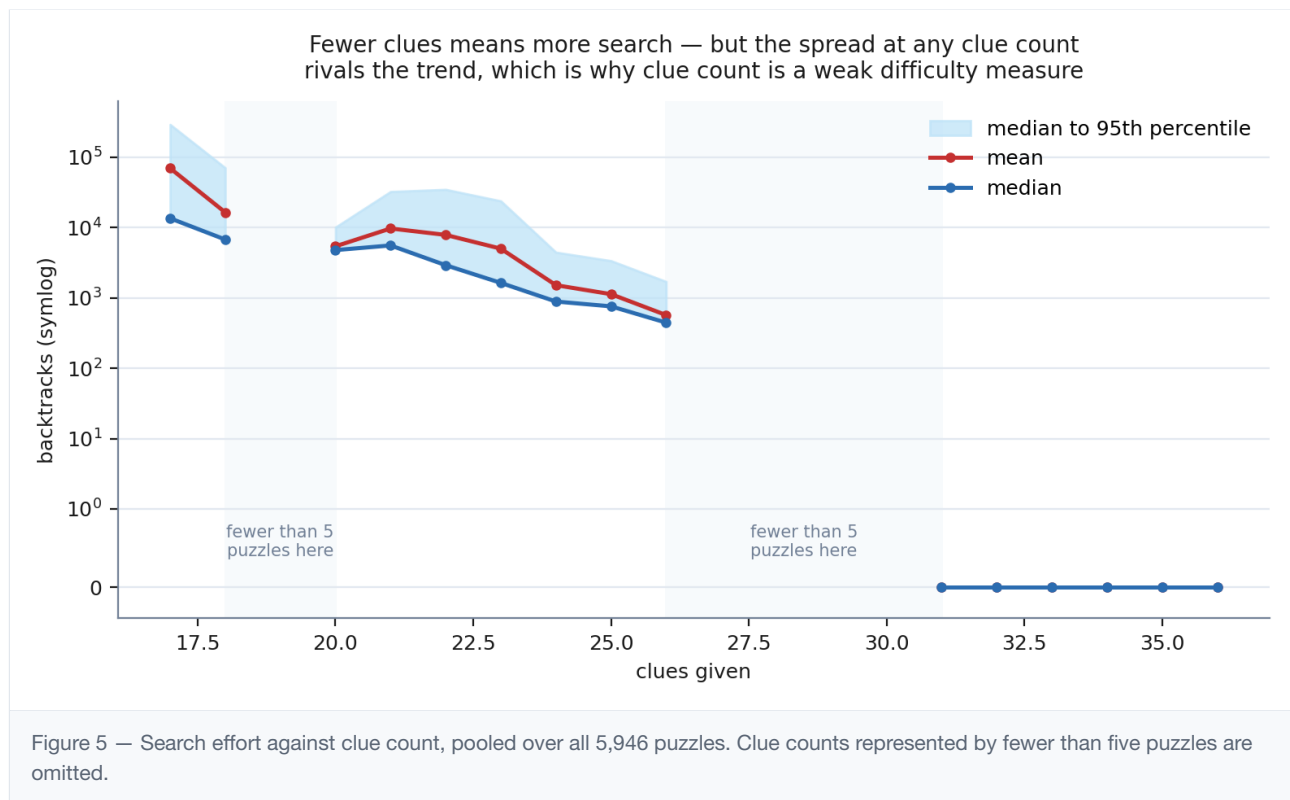
LEVEL	TARGET CLUES	MEAN CLUES	UNIQUE SOLUTION	NEEDED 0 BACKTRACKS	MEDIAN BACKTRACKS	MEAN	MAX
easy	45	45.0	30/30	30/30	0	0.0	0
medium	34	34.0	30/30	27/30	0	0.6	12
hard	26	26.0	30/30	4/30	32	73.1	277

Uniqueness is exact: 90 of 90 puzzles have precisely one solution. This is guaranteed by construction rather than by filtering, and it is the generator's core correctness property.

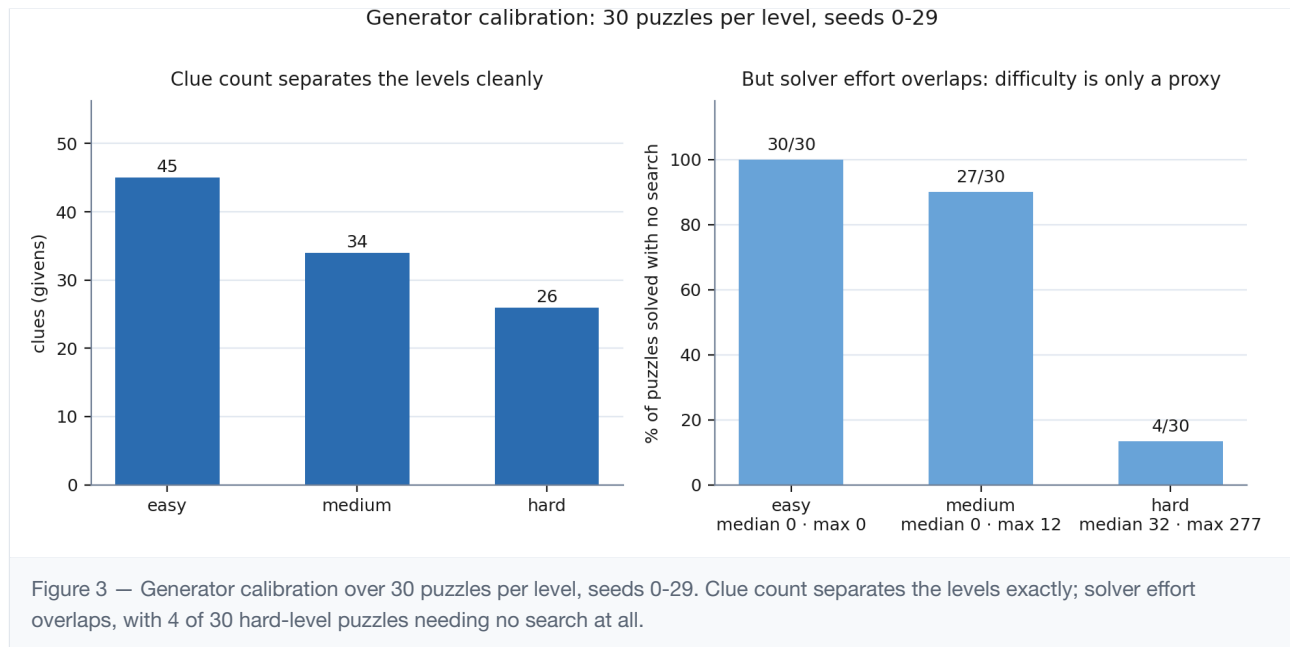
Clue count separates the levels perfectly, and difficulty only partly. The three levels hit their clue targets exactly, and mean effort does increase monotonically. But the *distributions overlap*: 4 of the 30 "hard" puzzles need zero backtracking, meaning they are solved by pure propagation — no harder, by this measure, than an "easy" puzzle. The hard level spans 0 to 277 backtracks. Clue count is therefore a real but weak signal, exactly as our progress report suspected.

Figure 5 makes the same point on the public data at much larger scale. Pooling all 5,946 puzzles by clue count, mean effort falls from 69,425 backtracks at 17 clues to 569 at 26 clues, and then to **exactly zero for all 1,998 puzzles with 31 or more clues** — not a small mean, a maximum of zero. (The 27–30 clue range contains only 10 puzzles across these sets, too few to read a trend from, so Figure 5 omits clue counts represented by fewer than five puzzles.) The trend is real and strong.

But the *spread within a single clue count* rivals the trend: at 17 clues the median is 13,488 backtracks and the 95th percentile is 290,904 — a factor of 22 among puzzles that are identical in the only property clue count can see.



This is why `rate_difficulty` rates by measured solver effort rather than clue count. It is an honest operational definition — it measures what our solver actually has to do — but we are clear about its limitation: **it is a measure of machine difficulty, not human difficulty**. A puzzle requiring a long forcing chain that a human finds agonising may need few backtracks, and the techniques a human uses (X-wing, colouring) are not the techniques our solver uses. We did not attempt to validate our rating against human solving times, and we do not claim it predicts them.



3.6 Qualitative evaluation

The visualisation is the evidence that the solver reasons the way we claim. `solve_smart` emits its own decision trace, and both front ends replay it, so what a viewer sees is the actual search that produced the numbers above, not a reconstruction. Two traces are worth watching:

- *An easy puzzle* — the MRV ring moves from forced cell to forced cell, candidate marks shrink monotonically, and no cell is ever un-filled. This is what "0 backtracks" looks like: a chain of deductions with no guessing.
- *AI Escargot* — the ring settles on a cell with two or three candidates, one is placed, peers prune, and after a few levels the trace unwinds in red as the branch dies. This is what search looks like when propagation is insufficient, and it is the visual counterpart of the 151 backtracks in the table.

Sanity checks, all passing. An already-solved grid returns in 1 node and 0 backtracks. An easy puzzle solves with 0 backtracks. An empty grid is correctly reported as having multiple solutions rather than one. A grid with duplicate givens is flagged by `conflicts` before solving. A grid that is legal but unsatisfiable returns no solution rather than an invalid one. All five are automated in `tests.py`.

Regression suite. 24 tests covering peer symmetry, parsing, all three solvers, agreement between solvers on 40 hard puzzles, the node cap, uniqueness, clue targets, seed reproducibility, and the difficulty rating. All pass.

3.7 Threats to validity and limitations

We would rather state these than have them found.

1. **Timing measurements are weak.** Taken on a loaded shared machine (load average 13–19), where repeated runs of the same deterministic solve varied by up to $8\times$. Node and backtrack counts are unaffected and exact; every time figure should be read as indicative only. This is why search effort, not time, is our primary metric.
2. **Capped baseline comparisons are lower bounds.** Wherever plain backtracking hit its node cap we report `>` and refuse to compute a ratio as though it were measured. The honest statement about the 17-clue preset is "the baseline had not finished after 10 million nodes," not "the speedup is $2,431\times$."
3. **Large sets are sampled, not exhausted.** `kaggle` and `minimal17` are sampled at 2,000 puzzles with a fixed seed. Sampling is random and reproducible, but a 2,000-puzzle sample cannot rule out a rare pathological instance in the remaining 145,000.
4. **The three-way solver comparison uses smaller samples** (95–150 per set) than the solve-rate experiment, because running both solvers on every puzzle is expensive. The node reductions are large enough that sampling error cannot plausibly reverse their direction, but the precise multipliers carry more uncertainty than the solve rates do.
5. **Difficulty is machine difficulty.** See 3.5. Not validated against human solvers.
6. **Propagation still stops short of the state of the art.** We implement forward checking, naked singles, and hidden singles. We do not implement naked/hidden pairs and triples, box-line reduction, X-wing, or full arc consistency, and we do not use a SAT or exact-cover encoding (Algorithm X / DLX), which is how the fastest published Sudoku solvers work. Our worst case is 2.46 million backtracks; a solver with stronger inference would cut that substantially.
7. **Generated puzzles are not minimal.** Removal follows a random order and stops at a clue target, so the result is irreducible with respect to that order, not globally minimal.

3.8 Summary of results against our stated goals

GOAL (FROM PROPOSAL / PROGRESS REPORT)	OUTCOME
Solve any valid partial grid	100% solve rate, verified, on 5,946 puzzles from 6 public sets
10–100× backtrack reduction vs. baseline on hard puzzles	Met and exceeded: baseline fails to finish 10M nodes where the solver takes 4,113
Generate puzzles with a guaranteed unique solution	90/90 generated puzzles verified unique, by construction
Difficulty rating	Implemented as measured solver effort; calibrated and shown to be a better signal than clue count, with limits stated
Visualise the search	Terminal and browser, both driven by the solver's own event trace
Moonshot: hardest known puzzles in well under a second	Met. AI Escargot in 0.63 ms; the engineered 17-clue puzzle in 11–19 ms with FC+MRV (range across runs, machine loaded) and 0.38 ms with stronger propagation
Stretch goal: stronger propagation	Done. Hidden singles: up to 12,827× fewer nodes, 150× faster on 17-clue puzzles — and 1.45× <i>slower</i> on easy ones, which is the more interesting finding

The shortcomings our progress report listed for itself — evaluation resting on three puzzles, difficulty defined only by clue count, propagation stopping at forward checking — are all addressed above.

Figure index

All figures appear inline above and are generated by `figures.py` from `results/results.json`. No figure recomputes a value, so figures and tables cannot disagree.

FIGURE	FILE	SHOWS
1	<code>results/fig1_peers.png</code>	The constraint structure: one cell and its 20 peers
2	<code>results/fig2_presets.png</code>	Search effort, informed vs. uninformed, on the presets (log scale)
3	<code>results/fig3_generator.png</code>	Generator calibration: clue count separates levels, effort overlaps
4	<code>results/fig4_datasets.png</code>	Solve rate and effort distribution across all six benchmark sets
5	<code>results/fig5_scaling.png</code>	Search effort vs. clue count, pooled over 5,946 puzzles
6	<code>results/fig6_crossover.png</code>	The cost/benefit crossover of stronger propagation

4 References and Resources Used

Algorithms and methods

1. **S. Russell and P. Norvig**, *Artificial Intelligence: A Modern Approach*. The constraint satisfaction chapter is the source of the standard formulations we implement: backtracking search, forward checking, the Minimum Remaining Values heuristic, and the fail-first rationale for it. Our decision not to add Least Constraining Value is a departure from the MRV+LCV pairing recommended there, justified empirically in Section 2.4.
2. **P. Norvig**, "Solving Every Sudoku Puzzle," <https://norvig.com/sudoku.html>. A well-known reference implementation of constraint propagation plus depth-first search for Sudoku. It informed our peer-based pruning and our use of the naked-single / hidden-single pair as the propagation fixpoint in `solve_strong`. No code was copied; the design ideas are credited here.
3. **Course materials for CS 175**, in particular the framing of puzzles as tasks with no environmental feedback, where success comes from search and inference rather than from a reward signal. This is the basis of the modelling argument in Section 1.

Benchmark data

All puzzle data is third-party and public. None of it was created by us, with the sole exception of the puzzles produced by our own generator in Section 3.3.

1. **puzzles/top95.txt** — 95 hard puzzles, collected by Peter Norvig, <https://norvig.com/top95.txt>. Named in our proposal as a stress-test set.
2. **puzzles/hardest11.txt** — 11 "hardest" puzzles, collected by Peter Norvig, <https://norvig.com/hardest.txt>.
3. **data/kaggle100k.txt** — a 100,000-puzzle export of the Kaggle *1 million Sudoku games* dataset (originally <https://www.kaggle.com/bryanpark/sudoku>), named in our proposal as the primary source for broad solve-rate and timing statistics.
4. **data/minimal17.txt** — 49,158 minimal 17-clue puzzles, originating from Gordon Royle's collection of 17-clue Sudoku grids and later extensions. 17 clues is the proven minimum for a uniquely-solvable Sudoku, which makes this the most adversarial large set available.
5. **data/top1465.txt** — 1,465 hard puzzles, the "top1465" list from <http://magictour.free.fr/sudoku.htm>.
6. **data/forum_hardest1106.txt** — a forum-curated collection of puzzles selected for difficulty against solvers.

Sets 6–9 are distributed together in the data archive of the **tdoku** Sudoku-solver benchmark project by T. Dillon, <https://github.com/t-dillon/tdoku>. `fetch_data.py` downloads that archive and extracts these four files, so the data step of our evaluation is reproducible by running one script.

1. **AI Escargot**, by Arto Inkala (published 2006), used as our named hard single instance. It is included in the repository as the `escargot` preset and as `puzzles/escargot.txt`. The "AI" in the name is the author's initials, not a reference to artificial intelligence.

Software

1. **Python 3 standard library only** for the solvers, generator, and command-line tool — `random`, `time`, `dataclasses`, `os`, `argparse`, `json`, `statistics`, `urllib`, `zipfile`. No third-party AI or ML packages are used anywhere in the system.
2. **unittest** (standard library) for the 24-case regression suite in `tests.py`.
3. **matplotlib** for the figures in this report, used only in `figures.py`, which reads `results/results.json` and draws it. No figure recomputes a result, so the figures cannot drift from the tables.

Reproducing everything in this report

```
python3 fetch_data.py      # download the public benchmark sets into data/
python3 tests.py           # 24-case regression suite
python3 benchmark.py all   # every experiment -> results/results.json
python3 figures.py         # every figure in this report, from that JSON
```

Node and backtrack counts are deterministic and will reproduce exactly. Wall-clock timings will not: they depend on the machine and its load at the time, which is why search effort rather than time is our primary metric, and why Section 3 reports the measurement conditions alongside the timings.

Statement of original work

The CSP formulation, all three solvers, the uniqueness counter, the generator, the difficulty rating, the terminal and browser visualisations, the test suite, the evaluation harness, and the figure code are our own work for this course. The algorithms they implement are standard and are credited above. The puzzle data is third-party and public, as itemised in references 4–10.